Dynaswarm

Recency-Driven Compute Allocation in Hybrid Swarm Metaheuristics for Dynamic Path Optimisation

Technical Report

Vaibhav Krishna Bansal
Rudra Rajpurohit
Kartikey Goyal
Pavit Agrawal

School of Computer Engineering, MIT Manipal (MAHE)
Mentor: Mrs. Vidhya V

November 2026

Abstract

Dynaswarm studies how a fixed per-step search budget should be divided between ant colony optimisation (ACO) and particle swarm optimisation (PSO) when a shortest path is sought on a graph that changes during the search. ACO keeps its memory as pheromone on edges, which goes stale when edges close or grow expensive. The PSO used here ranks nodes and keeps no edge memory. Every budgeted method receives 300 candidate-route constructions per timestep, runs on the same pre-generated sequence of graph states and is scored against the exact A* optimum at every step. A pre-registered run of 1,200 trials covered 150 paired seeds, four disruption rates, two graph sizes and ten algorithm arms, and recorded 720,000 step rows. The 50/50 hybrid beat PSO in all eight rate-by-size cells, by 0.73 to 0.92 percentage points (pp) of mean excess at 1,000 nodes. It beat ACO in all four cells at 1,000 nodes, by 10.65 to 11.08 pp with Cliff's delta near −0.99, but in only one of four cells at 200 nodes. Adapting ACO's evaporation rate produced no significant difference, although the widest 95% interval still allows a benefit of 1.07 pp. Moving the split in response to a disruption-recency signal did not beat a frozen split, apart from one cell with a negligible effect. At 1,000 nodes under the highest rate, a timetable that ignores the graph beat both sensing policies by a significant but negligible margin. The sensing policies read when the graph last changed and not how much it changed, so the run bounds recency-driven allocation and leaves magnitude-aware allocation untested.

Index Terms: dynamic path planning, ant colony optimisation, particle swarm optimisation, hybrid metaheuristics, compute allocation, pre-registration, reproducible experimentation.

Contents

1 Introduction and Research Objectives **3**
1.1 Problem context . . . 3
1.2 Research gap . . . 3
1.3 Objectives and claims . . . 3
1.4 Contributions and organisation . . . 4

2 Literature Survey: Research Published from 2023 to 2026 **4**
2.1 Selection and comparison rules . . . 4
2.2 Synthesis . . . 5

3 Algorithmic Foundations **5**
3.1 A*: exact per-timestep reference . . . 5
3.2 MAX-MIN Ant Colony Optimisation . . . 6
3.3 Priority-vector Particle Swarm Optimisation . . . 6
3.4 Complementarity . . . 6

4 Hybrid Architecture and Recency Control **7**
4.1 Allocation policies . . . 8
4.2 Design invariants . . . 8

5 Methodology **8**
5.1 Dynamic-world construction and experimental factors . . . 8
5.2 Fairness and randomisation . . . 9
5.3 End-to-end pseudocode . . . 9
5.4 Comparison family and pre-registration . . . 10
5.5 Recovery definition . . . 11
5.6 Statistical analysis plan . . . 11
5.7 Reproducibility . . . 11

6 Software Implementation and Dashboard Evidence **12**
6.1 System structure 12
6.2 Live route and result inspection 12

7 Results and Statistical Analysis **13**
7.1 Run integrity and mean excess 13
7.2 Claim 1: the hybrid against its components 14
7.3 Claim 2: adapting the evaporation rate 14
7.4 Claim 3: allocation control 14
7.5 Exploratory analyses 15

8 Discussion and Methodological Comparison **17**
8.1 The hybrid result 17
8.2 Precision of the null results 17
8.3 Explanations for the allocation null 17
8.4 Comparison with recent studies 18
8.5 Iterations, compute and parameter isolation 19

9 Individual Contributions and Project Governance **19**
9.1 Governance and quality assurance 19

10 Limitations and Threats to Validity **19**
10.1 Internal and statistical validity 19
10.2 Construct validity 20
10.3 External and reporting validity 20

11 Conclusion and Future Work **20**

Declarations **21**

References **21**

1 Introduction and Research Objectives

1.1 Problem context

Routes in road and communication networks are computed while the network changes. Congestion alters traversal cost, roads close and reopen, and new links appear, so the cheapest route at one timestep can be expensive or invalid at the next. At timestep t Dynaswarm models the network as

$$G^{(t)} = \big(V, E^{(t)}, w^{(t)}\big). \tag{1}$$

Given a source s and a target q, a method returns a simple feasible path $\hat{\pi}^{(t)}$ whose cost is

$$c\big(\hat{\pi}^{(t)}\big) = \sum_{(u,v)\in\hat{\pi}^{(t)}} w^{(t)}(u,v). \tag{2}$$

Raw cost cannot be compared across timesteps, because the optimum itself moves. Quality is therefore measured as the excess over the exact optimum $c^*(G^{(t)})$, which A* computes at every step [1]:

$$E^{(t)} = 100\,\frac{c(\hat{\pi}^{(t)}) - c^*(G^{(t)})}{c^*(G^{(t)})}. \tag{3}$$

Zero means the current optimum was found. Lower is better, and differences between methods are given in percentage points (pp) of excess.

Exact recomputation solves the routing problem itself. A* is exact at every step, and incremental methods such as Lifelong Planning A* and D* Lite repair the previous search instead of repeating it [2, 3]. The dynamic shortest path is used here as a controlled setting in which the true optimum is known at every step. The study concerns budget-limited approximate search inside that setting. Such methods hold a complete route after any amount of effort and improve it as effort grows, which is what allows one budget to be divided between two of them.

1.2 Research gap

The path-planning studies surveyed in Section 2 report shorter paths or fewer iterations on selected maps. An ACO iteration and a PSO iteration do not perform equal work, so equal iteration counts do not mean equal effort. Comparisons also become optimistic when methods see different random worlds, when dependent timesteps are counted as independent samples, or when many tests are read without correction for multiplicity. Dynaswarm fixes the number of candidate constructions per step, generates every world before any method runs, pairs trials by seed and reduces each dependent trajectory to one value per trial.

1.3 Objectives and claims

O1. Build a fair, reproducible dynamic-routing testbed. Every method sees the same graph sequence, draws its own random stream and spends the same number of candidate constructions.

O2. Measure what an ACO-PSO hybrid gains over its components under that budget, at two graph sizes and four disruption rates.

O3. Test whether moving the hybrid's budget split in response to disruptions improves on a frozen split and on a timetable that moves the split without sensing.

The objectives were tested as three claims, fixed before the evidence run:

Claim 1. A 50/50 ACO/PSO hybrid finds cheaper routes than either component alone under the same budget.

Claim 2. Adapting ACO's evaporation rate at run time does not improve on the same colony with the rate held fixed.

Claim 3. Moving the hybrid's split in response to how recently the graph was disrupted improves on a frozen split and on a timetable.

Claim 3 is the thesis. Claim 2 is its premise. Allocation is proposed as an alternative to adapting a component's parameters, so the parameter route is tested on the same footing.

1.4 Contributions and organisation

The project contributes a metered harness in which every method sees identical graph states and spends identical effort, a pre-registered family of nine paired comparisons, and the completed 1,200-trial run with confidence bounds on every null result. A dashboard replays graph events, budgets and routes step by step. Section 2 surveys recent work. Sections 3 to 5 describe the algorithms, the hybrid and the experimental method, and Section 6 the software. Section 7 gives the results and Section 8 interprets them. Sections 9 to 11 cover contributions, limitations and conclusions.

2 Literature Survey: Research Published from 2023 to 2026

2.1 Selection and comparison rules

The survey covers ten studies published from 2023 to 2026 that apply ACO, PSO or a hybrid built on one of them to robot path planning, and that state their representation, baseline and at least one outcome (Table 1). Each number is quoted in its own paper's benchmark. Maps, robot models, stopping rules and outcomes differ, so the numbers are not comparable with each other or with Dynaswarm. Where a paper does not report a quantity, the table says nothing about it.

Table 1: Studies from 2023 to 2026, with Dynaswarm last.

Year	Study	Method	Reported result
2023	Zhang et al., HACO [4]	ACO hybridised with chaotic-map PSO. Initial pheromone follows obstacle density, the heuristic follows heading, and PSO's search shapes the pheromone update.	Compared with ACO variants on several grid maps.
2023	Yang et al., IPSO-IDWA [5]	PSO with a modified velocity update and inertia weight supplies the global path to a dynamic window approach (DWA) extended with velocity obstacles.	Simulations of global planning and obstacle avoidance.
2024	Shan et al., IAACO-DWA [6]	Adaptive heuristic and evaporation, elite MAX-MIN updates and path withdrawal, combined with an improved DWA.	Against ACO on 20×20 and 30×30 maps, path length 10.1% and 13.7% shorter and iterations 63.3% and 63.0% fewer.
2024	Wang et al., improved ACO [7]	Potential-field-guided heuristic, best-worst pheromone increment and triangle pruning.	20×20 map: 31.17 m in 23 iterations to 28.96 m in 5. 30×30 map: 66.18 m in 33 iterations to 43.46 m in 15.
2024	Dao et al., MOPSO-DWA [8]	Multi-objective PSO chooses length-smoothness trade-offs and DWA reacts to unknown obstacles.	Over 20 runs, 97% goal reaching. Baselines include A*-DWA, NSGA-II and SPEA.
2025	Ma et al., ITACO [9]	Dynamic transition weighting, trimming of failed ants and path simplification.	Path length 6.68% shorter than the latest related results, as stated in the abstract.

Year	Study	Method	Reported result
2025	Li et al., IEACO [10]	Non-uniform initial pheromone, ϵ-greedy moves, adaptive exponents, a multi-objective heuristic and dynamic evaporation.	Simple map: mean 29.78 (SD 0) in 6.5 iterations. Complex map: mean 28.63 (SD 0) in 4.5 iterations.
2026	Ahmad et al., ABGPSO [11]	Sigmoid time-varying coefficients steer PSO between exploration and exploitation.	Four static layouts: travel time reduced by up to 69%, 67%, 72% and 79% against the compared metaheuristics.
2026	Huang et al., IACO-DWA [12]	Gravity and direction terms in ACO supply subgoals to an improved DWA.	Four static maps: path length reduced by up to 30.03% and turns by up to 71.43% against ACO. Static and dynamic obstacles avoided.
2026	Zhang et al., IACO-QDWA [13]	ACO with positive and negative feedback, and a DWA tuned by Q-learning among dense dynamic obstacles.	Reports the balance between path quality, avoidance success rate and real-time performance.
2026	Dynaswarm (this report)	MAX-MIN ACO and node-priority PSO share a metered budget on a changing graph. Recency controls the split. A* gives the exact optimum at each step.	150 paired seeds, two graph sizes, four disruption rates, 60 steps, ten arms, 720,000 step rows. Wilcoxon tests with Holm correction across 34 cells, Cliff's delta, bootstrap bounds and recovery analysis.

2.2 Synthesis

The 2023 studies combine ACO or PSO with a second method, PSO in one and a dynamic window planner in the other. The 2024 studies add adaptive pheromone control, pruning and multi-objective local planning. The 2025 and 2026 studies refine the balance between exploration and exploitation and handle dynamic obstacles, and some report repeated runs. Progress is usually measured in iterations. None of the ten reports, as far as their text shows, counts work in a unit shared across methods.

Dynaswarm does not claim a shorter route than any of these studies, and it does not model robot motion. Its graphs change through edge events rather than moving obstacles. Its contribution is the evaluation: identical changing worlds, an exact moving optimum, equal work, paired trials, multiplicity correction and recovery analysis in one design. The robot-planning studies address kinematics, smoothing and collision avoidance, which Dynaswarm leaves out, so the two lines of work answer different questions.

3 Algorithmic Foundations

3.1 A*: exact per-timestep reference

A* expands nodes in order of $f(v) = g(v) + h(v)$, where g is the cost from the source and h an admissible estimate of the cost still to go, and it returns a shortest path [1]. Edge cost grows linearly with Euclidean length here, so straight-line distance to the target is admissible. A* is rerun from scratch at every graph state and supplies the denominator of Eq. (3). It is a reference and not a budgeted arm, and the approximate methods are compared with each other. A claim about routing speed would need an incremental exact method such as D* Lite [3] as its baseline, and this report makes no such claim.

3.2 MAX-MIN Ant Colony Optimisation

An ant at node u moves to an active, unvisited neighbour v with probability

$$P(u \to v) = \frac{\tau_{uv}^{\alpha}\, \eta_{uv}^{\beta}}{\sum_{z \in N^{+}(u)} \tau_{uz}^{\alpha}\, \eta_{uz}^{\beta}}, \qquad \eta_{uv} = \frac{1}{w_{uv} + \gamma\, h(v)}, \tag{4}$$

where τ is pheromone and the visibility η combines edge cost with straight-line distance to the target [14]. Pheromone evaporates as $\tau \leftarrow (1-\rho)\tau$, the best route deposits new trail, and trail is clamped to $[\tau_{\max}/2n,\ \tau_{\max}]$ with $\tau_{\max} = 1/(\rho\, C_{\text{best}})$, where n is the number of nodes and C_{best} the best cost found [15]. The defaults are $\alpha = 1$, $\beta = 2$, $\rho = 0.10$, $\gamma = 1$ and 20 ants per iteration.

When the graph changes, trail on edges that have become inactive is set to zero and trail on surviving edges is kept. That surviving trail helps while the graph is stable. Just after a change it can mislead, since it still favours the neighbourhood of a route that has become worse. Mavrovouniotis and Yang adapted the evaporation rate during dynamic routing problems [16], and Claim 2 tests that parameter route on the same footing as allocation.

3.3 Priority-vector Particle Swarm Optimisation

PSO moves real-valued vectors under inertia and under attraction to each particle's own best position and to the swarm's best [17]. Here each particle i holds one priority per node, $x_i \in [0,1]^{|V|}$, and evolves as

$$v_i^{k+1} = \omega v_i^{k} + c_1 r_1 \big(p_i - x_i^{k}\big) + c_2 r_2 \big(g - x_i^{k}\big), \tag{5}$$

$$x_i^{k+1} = \operatorname{clip}\big(x_i^{k} + v_i^{k+1},\, 0,\, 1\big), \tag{6}$$

where p_i is the particle's best position, g the swarm's best and r_1, r_2 uniform random draws. A route is decoded greedily from the source. At each step the walk takes the unvisited neighbour with the largest product of priority and the visibility η of Eq. (4). A particle that reaches a dead end is repaired by perturbing its priorities, up to three times, and each repair is charged to the budget. The defaults are $\omega = 0.7$, $c_1 = c_2 = 1.5$ and 20 particles.

When the graph changes, particle positions are kept, since they rank nodes and the nodes are unchanged. Each particle's remembered best cost is discarded and re-established on the new graph, and the swarm's best route is re-priced and kept only if it is still feasible. PSO therefore has no edge trail to go stale. The encoding is specific to this project, and results about it are not results about PSO routing in general.

3.4 Complementarity

The two methods keep memory in different places. Just after a change, ACO's surviving trail can point at a route that has become worse, while PSO's node priorities remain meaningful. In calm periods the trail concentrates the colony on a good route, which the greedy PSO decoder does less well. The design premise is that the two methods err at different times, so a hybrid that shares one budget could move effort toward PSO after a change and back toward ACO in calm periods.

Table 2: Role of each algorithm in the study.

Algorithm	Memory	Strength	Weakness
A*	none, exact search	exact optimum at every step	not budgeted, so not ranked
MAX-MIN ACO	pheromone on edges	refinement in stable periods	surviving trail misleads after a change
Priority PSO	priorities on nodes	no edge memory to go stale	greedy decoding dead-ends and refines less
Hybrid	both, one budget	better of two routes, shared information	the split may not suit the moment

4 Hybrid Architecture and Recency Control

The hybrid has one budget of $B = 300$ candidate constructions per step. A controller sets the ACO share f_t, and the budget is split as

$$B_{\mathrm{ACO}}^{(t)} = \max\bigl(1, \mathrm{round}(f_t B)\bigr), \qquad B_{\mathrm{PSO}}^{(t)} = \max\bigl(1, B - B_{\mathrm{ACO}}^{(t)}\bigr). \tag{7}$$

Both components search the same graph state with their own random streams. The hybrid re-prices both routes on the current graph and returns the cheaper one. After each step the components exchange their best routes. ACO's best raises PSO's priorities along that route, and PSO's best deposits trail for ACO. Changing f_t moves constructions between the components without changing their total (Fig. 1).

The controlled hybrids sense disruption through a recency signal,

$$r_t = \begin{cases} 1 & \text{if a disruption landed at step } t, \\ 0.7\, r_{t-1} & \text{otherwise,} \end{cases} \tag{8}$$

which jumps to 1 when the graph changes and decays geometrically while it stays calm. The policy reads the signal after the step's search, so a disruption at step t changes the split used from step $t + 1$.

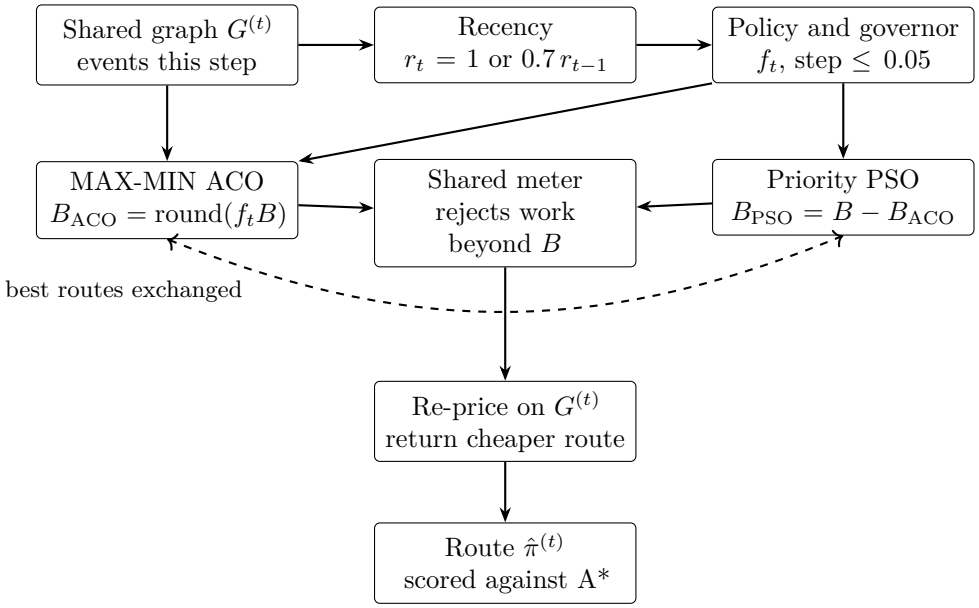


Figure 1: One step of a controlled hybrid. The graph state and the budget are shared. Each component keeps its own random stream, and the two exchange their best routes after each step. The share that the policy sets after a step's search is used at the next step.

4.1 Allocation policies

Four policies set f_t (Table 3). The scheduled policy is the control for the other two moving policies. It requests the same span of shares as the direct policy, from 0.8 to 0.2, but ignores the graph, so it separates the effect of moving the split from the effect of sensing when to move it.

Table 3: Allocation policies for the controlled hybrids.

Policy	Requested ACO share f_t
frozen	0.5 at every step
incremental	moves by 0.006 per step, toward PSO when r_t is above a slowly updated baseline and toward ACO when it is below
direct	$0.5 + 0.6\,(0.5 - r_t)$, so 0.8 when calm and 0.2 on a disruption step
scheduled	linear sweep from 0.8 at $t = 0$ to 0.2 at $t = 59$, independent of the graph

A governor applies every request. It limits the change in f_t to 0.05 per step, keeps f_t within $[0.05, 0.95]$, and records each request it shortens as a clamp. The two sensing policies take all of their disruption information from r_t. The number of edge events at the step is also available to them and is not read, so a disruption that changed a few edges and one that changed dozens draw the same response.

A separate controller drives the adaptive ACO arm used for Claim 2. It raises or lowers ρ by 0.001 per step as the fraction of the last ten steps that carried a disruption moves above or below its own slowly updated baseline.

4.2 Design invariants

Three conditions hold at every step. Component budgets sum to B, and the meter that counts them is created by the harness, so no method can change its own allowance. No returned route may use a closed or missing edge. A route found earlier is re-priced on the current graph rather than trusted at its old cost, because an unchanged sequence of nodes can become more expensive or infeasible.

5 Methodology

The design aims to make two arms compared on the same seed differ only in the mechanism under test. Each subsection below fixes one more source of difference.

5.1 Dynamic-world construction and experimental factors

Graphs are random geometric graphs. Nodes are placed in the unit square and joined within a radius set by a density parameter, and disconnected components are joined during construction. Two sizes, called tiers, are used: `medium200`, with 200 nodes at density 0.04, and `large1000`, with 1,000 nodes at density 0.01. The source is node 0 and the target is the node farthest from it among those reachable.

Each trial lasts 60 timesteps. The number of disruptions at a step is Poisson-distributed with an absolute rate of 0, 0.1, 0.3 or 0.8 per step, in the scenarios `calm`, `light`, `moderate` and `severe`. A step is therefore quiet with probability 100%, 90%, 74% and 45%. The rate does not scale with graph size, so the larger graph receives no more disruptions per step than the smaller one. Each disruption draws a spatially clustered set of up to 25 edge events. The event kinds are closure, deletion, a cost increase or decrease by a factor of two, with decreases floored at the free-flow cost, reopening of an original edge, and addition of an edge from a fixed pool. An event that would disconnect the source from the target is rejected.

The full event schedule is generated before any search method is built. The graph at step t is a function of the base graph and the earlier events alone, so the order in which arms run cannot change the world any of them sees. Table 4 summarises the design, and Table 5 lists the arms.

Table 4: Factorial design of the evidence run.

Item	Setting
Graph tiers	`medium200`: 200 nodes, density 0.04. `large1000`: 1,000 nodes, density 0.01
Scenarios	`calm`, `light`, `moderate`, `severe`: 0, 0.1, 0.3, 0.8 disruptions per step
Disruption size	up to 25 edge events per disruption, at both tiers
Trial length	60 dependent graph states
Seeds	150 paired seeds (1000 to 1149) in every tier and scenario
Arms	ten (Table 5)
Budget	300 candidate constructions per step for every arm except A*
Run size	1,200 trials and 720,000 step rows
Primary endpoint	mean excess per trial
Secondary metrics	worst excess, share of steps at the optimum, recovery after damage

Table 5: The ten arms. Arms in an ablation pair draw identical random numbers.

Arm	Role	Allocation or adaptation	Shares stream with
`astar`	reference	exact shortest path, not ranked	none
`aco`	component	ACO alone, full budget	none
`pso`	component	PSO alone, full budget	none
`hybrid`	Claim 1	50/50 split, no control machinery	none
`aco_fixed`	Claim 2 baseline	ACO with $\rho = 0.10$ fixed	`aco_adaptive`
`aco_adaptive`	Claim 2	ACO with ρ adapted to disruption frequency	`aco_fixed`
`hybrid_fixed`	Claim 3 baseline	frozen policy, control machinery present	other controlled hybrids
`hybrid_adaptive`	Claim 3	incremental policy on recency	other controlled hybrids
`hybrid_direct`	Claim 3	direct policy on recency	other controlled hybrids
`hybrid_scheduled`	Claim 3 control	timetable, ignores the graph	other controlled hybrids

5.2 Fairness and randomisation

All arms see the same graph state, checked by a hash at every step, and all are scored against the same optimum. The budget is counted in candidate constructions by a meter outside each method, and repair attempts count like any other construction. Counting constructions stops a larger colony or swarm from buying extra work, and it makes the comparison independent of machine speed, which an iteration or wall-clock budget would not. Random streams are derived from the seed and the arm's family name, so an arm's random numbers depend on nothing else, and adding an arm does not alter another arm's trajectory. All arms use the default settings of Section 3, and no parameter was tuned. The trial is the experimental unit, and its 60 dependent timesteps are reduced to one value before any test.

5.3 End-to-end pseudocode

Listing 1 gives one paired trial and Listing 2 one step of a controlled hybrid.

Listing 1: One paired dynamic trial.

```
Input: seed s, tier, scenario, arms A, budget B = 300, horizon T = 60
graph0   <- generate_graph(stream(s, "graph"), tier)
events   <- draw_events(stream(s, "events"), graph0, scenario, T)
timeline <- replay(graph0, events)          # fixed before any arm runs
for each arm a in A:
    a.initialise(stream(s, family(a)))      # own random stream
for t = 0, ..., T-1:
    G      <- timeline.state_at(t)
    c_star <- A_star(G)                     # exact optimum
    for each budgeted arm a in A:
        a.rebind(G)                         # drop closed-edge trail etc.
        meter <- Meter(limit = B)           # created by the harness
        route <- a.step(meter)
        if route exists:
            excess[a, t] <- 100 * (cost(route, G) - c_star) / c_star
        log(s, t, a, route, c_star, events_at(t), meter.count)
reduce each trajectory to trial values; compare arms paired by seed
```

Listing 2: One step of a controlled hybrid.

```
Input: graph G at step t, budget B, share f set after step t-1
B_aco <- max(1, round(f * B));  B_pso <- max(1, B - B_aco)
p_aco <- ACO.step(G, meter.limit(B_aco))
p_pso <- PSO.step(G, meter.limit(B_pso))
ACO.deposit(p_pso);  PSO.attract(p_aco)                  # exchange
route <- the cheaper of p_aco and p_pso, re-priced on G
r <- 1 if a disruption landed at t, else 0.7 * r         # recency
f <- governor(policy(r, t), previous = f,                # |change| <= 0.05
              bounds = [0.05, 0.95])                     # used at t+1
return route
```

5.4 Comparison family and pre-registration

Nine paired comparisons were fixed before the run (Table 6). The six Claim 3 comparisons cover every pair among the four controlled hybrids, so no pairing could be chosen after the result. Nine comparisons in four scenarios give 36 cells per tier. In `calm` the P1 and P4 arms behave identically on every trial, since there is nothing to sense, which leaves 34 tested cells. `calm` is the negative control: a sensing policy that moves without disruption is responding to something else. The direct policy is the designed exception, holding f near 0.8 because $r_t = 0$ throughout.

Table 6: The pre-registered comparison family. Negative effects favour the first arm.

Code	Pair	Question
S2a	`hybrid` vs `aco`	Claim 1: does the hybrid beat ACO
S2b	`hybrid` vs `pso`	Claim 1: does the hybrid beat PSO
P4	`aco_adaptive` vs `aco_fixed`	Claim 2: does adapting ρ help
P1	`hybrid_adaptive` vs `hybrid_fixed`	does incremental sensing beat not moving
P2	`hybrid_direct` vs `hybrid_fixed`	does direct sensing beat not moving
P5	`hybrid_scheduled` vs `hybrid_fixed`	does moving without sensing help
P3a	`hybrid_adaptive` vs `hybrid_scheduled`	does incremental sensing beat the timetable
P3b	`hybrid_direct` vs `hybrid_scheduled`	does direct sensing beat the timetable
S1	`hybrid_direct` vs `hybrid_adaptive`	does the shape of the response matter

The decision rules for Claim 3 were written down with the family:

- if a sensing arm beats both `hybrid_fixed` and `hybrid_scheduled`, Claim 3 is supported.
- if it beats `hybrid_fixed` but not `hybrid_scheduled`, what helped was moving the split, not sensing when to move it.
- if neither sensing arm beats `hybrid_fixed`, the result is a null, reported with the bound the run places on any benefit.
- if `hybrid_direct` beats `hybrid_adaptive`, the shape of the response matters.

For Claim 2, no significant difference reproduces the claim, a significant advantage for the adaptive colony contradicts it, and one for the fixed colony means adaptation hurts. Three dated amendments added P4, restricted recovery to damaging disruptions and split the timetable comparison into P3a and P3b. The design was also widened from one scenario to four, with Holm correction across the full grid, and P5 was added. All of these changes preceded the run.

5.5 Recovery definition

Recovery is the number of steps an arm needs to bring its excess back within 10% of its level before a disruption, with that level taken as the mean of the two preceding steps, both free of disruption. Only disruptions that raised the arm's excess are scored. Disruptions with fewer than eight steps left in the trial are dropped, and one that never recovers is scored at the remaining horizon. A floor of 30 damaging events per arm, set before the run, decides where recovery can be reported. Recovery values were first tabulated after the run, so they appear with the exploratory results.

5.6 Statistical analysis plan

Every comparison is paired by seed, which removes the large differences in difficulty between random graphs. The Wilcoxon signed-rank test is applied to the 150 paired trial means [18]. Holm's procedure [19] controls the family-wise error once across the 34 tested cells of a tier. With p-values ordered $p_{(1)} \le \cdots \le p_{(m)}$, hypotheses are rejected in order while

$$p_{(j)} \le \frac{\alpha}{m - j + 1}, \tag{9}$$

with $\alpha = 0.05$ and $m = 34$, so the strictest threshold is about 0.0015. Effect size is Cliff's delta [20],

$$\delta = \Pr(X > Y) - \Pr(X < Y), \tag{10}$$

which lies between -1 and 1. Magnitudes below 0.147, 0.33 and 0.474 are read as negligible, small and medium, and larger ones as large [21]. For each pre-registered comparison that is not significant, a 95% percentile bootstrap interval on the mean paired difference, from 4,000 resamples of the seeds [22], gives the largest advantage the data allow. A significant result with a negligible $|\delta|$ is reported as detectable but practically small. Analyses chosen after the result was seen are labelled exploratory and are not used to overturn a pre-registered outcome.

5.7 Reproducibility

The evidence run used commit `b4d3d28` with a clean working tree, Python 3.13.0 and NumPy 2.5.2 on macOS (Darwin 25.0.0, arm64), with eight parallel jobs. Its provenance record lists all 150 seeds. At the run commit the repository held 536 Python and 53 JavaScript tests, and before the run one trial regenerated from the manifest matched its recorded rows bit for bit. The 720,000 step rows can be regenerated from the commit and the seed list, and every statistic except the timings can be recomputed from them.

6 Software Implementation and Dashboard Evidence

6.1 System structure

Graph generation and event replay are separate from the search methods. A fixed timeline supplies graph states, each arm rebinds to the current state, the meter enforces the allowance, and the analysis reduces the logged rows to paired trial values. The code uses NumPy and SciPy, pandas and PyArrow for the Parquet step records, Hydra for configuration and joblib for parallel trials. A FastAPI dashboard replays recorded trials and can run new ones, which are stored apart from the evidence run and treated as exploratory.

6.2 Live route and result inspection

Fig. 2 shows the configuration screen. It sets the graph, the seed, the disruption rate and size, the horizon, the arms and the common budget. The screenshot uses 200 nodes at density 0.04, seed 1000, the `moderate` rate, 25 edge events per disruption, 60 steps and 300 constructions, with A*, ACO, PSO and the plain hybrid selected.

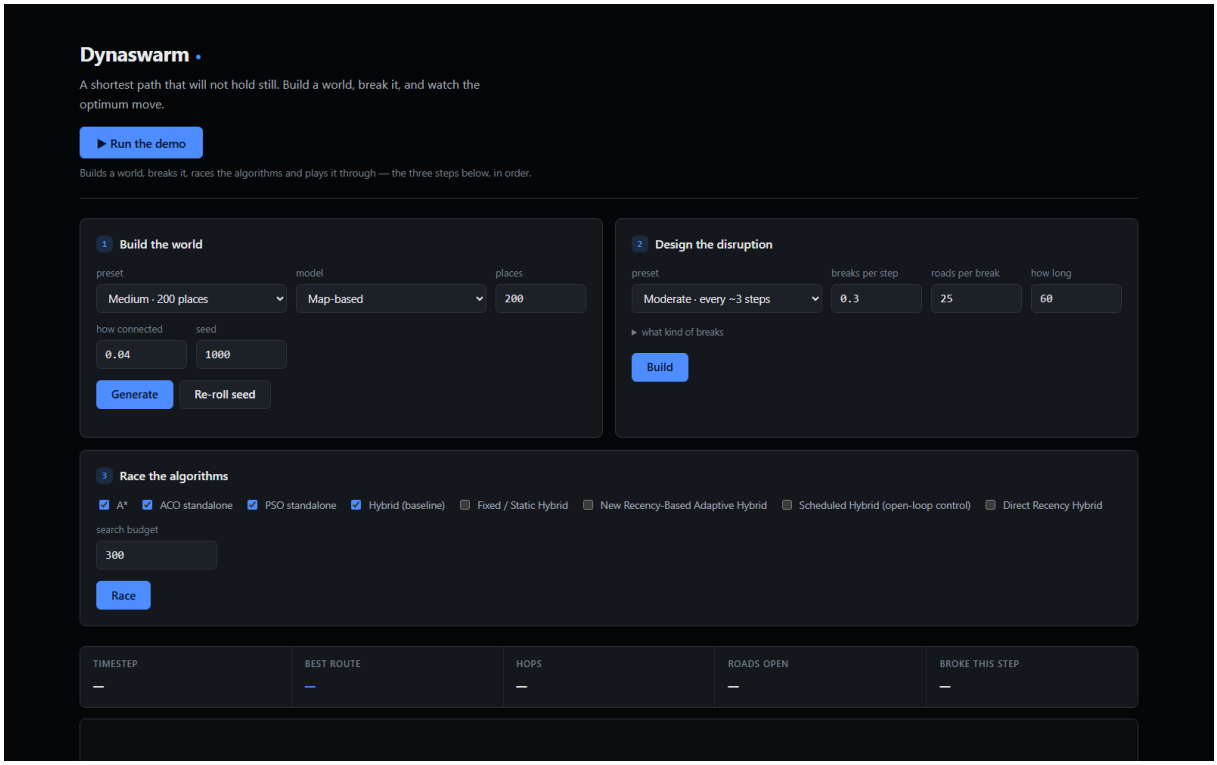


Figure 2: Dashboard configuration screen. The world, the disruption and the arms are chosen before the race.

Fig. 3 shows the result of that demonstration. All arms are drawn on the same graph state, and the chart keeps the step-by-step behaviour that a mean hides. The run gave mean excess of 0.0% for A*, 2.0% for ACO, 34.6% for PSO and 0.5% for the hybrid. These are one seed and carry no statistical weight, and the evidence in Section 7 comes from the recorded run alone.



Figure 3: Dashboard result view for the seed-1000 demonstration. Four panes show each arm's route on the same graph state, the chart shows excess over the moving optimum, and the banner marks the one-seed ranking as exploratory.

7 Results and Statistical Analysis

7.1 Run integrity and mean excess

The run completed 1,200 trials, with all ten arms in every trial. It recorded 720,000 step rows and took 216 minutes at eight jobs: 24.3 minutes at 200 nodes and 191.8 at 1,000. Every tested cell holds 150 paired seeds, and the two degenerate cells are exactly `calm`/P1 and `calm`/P4. Rerunning the analysis on the same rows reproduced every comparison.

Table 7 gives mean excess for every ranked arm. At 1,000 nodes every hybrid sits near 3% and both ACO variants near 13 to 14%. At 200 nodes the gap between the hybrid and ACO is small, and in `calm` it is reversed.

Table 7: Mean excess (%) per arm, scenario and tier: the mean over 150 seeds of each trial's mean excess, the primary endpoint. Steps on which an arm had no route are left out of its trial mean. Lower is better.

	`large1000`				`medium200`			
Arm	calm	light	moderate	severe	calm	light	moderate	severe
`aco`	13.33	13.53	13.94	14.28	4.93	6.11	8.49	10.49
`pso`	3.45	3.50	3.76	4.19	8.02	9.35	13.34	20.11
`hybrid`	2.68	2.77	2.85	3.27	5.29	5.54	7.49	8.48
`aco_fixed`	12.77	13.28	13.54	14.25	4.99	6.01	8.64	10.90
`aco_adaptive`	12.77	13.26	13.15	14.01	4.99	6.04	8.49	11.23
`hybrid_fixed`	2.58	2.69	2.86	3.15	4.84	5.62	7.58	8.98
`hybrid_adaptive`	2.58	2.71	2.95	3.27	4.84	5.37	7.69	9.23
`hybrid_direct`	2.44	2.51	2.79	3.24	4.16	5.58	7.20	9.48
`hybrid_scheduled`	2.48	2.61	2.76	2.99	4.39	5.46	6.90	8.68

Of the 34 tested cells per tier, 12 were significant after Holm correction at `large1000` and 5 at `medium200`. The three claims are reported in turn.

7.2 Claim 1: the hybrid against its components

The hybrid beat PSO in all four scenarios at both tiers (Table 8). Against ACO it won all four scenarios at 1,000 nodes, by 10.65 to 11.08 pp, with Cliff's delta between −0.989 and −0.993 and adjusted $p = 7.8 \times 10^{-25}$ in each cell. At 200 nodes it beat ACO significantly only in `severe`, by 2.01 pp ($\delta = -0.266$). By Cliff's delta the advantage over PSO is small, except at `medium200/severe`, where it is large.

Table 8: Claim 1. Mean paired difference in pp (hybrid minus component), Cliff's delta and Holm-adjusted p. Bold cells are significant.

		Hybrid vs ACO (S2a)			Hybrid vs PSO (S2b)		
Tier	Scenario	Diff.	δ	p_{Holm}	Diff.	δ	p_{Holm}
`large1000`	calm	**−10.65**	−0.989	7.8×10^{-25}	**−0.77**	−0.287	1.9×10^{-5}
	light	**−10.77**	−0.990	7.8×10^{-25}	**−0.73**	−0.263	8.9×10^{-5}
	moderate	**−11.08**	−0.990	7.8×10^{-25}	**−0.90**	−0.311	3.9×10^{-7}
	severe	**−11.01**	−0.993	7.8×10^{-25}	**−0.92**	−0.240	1.5×10^{-4}
`medium200`	calm	+0.36	−0.107	1.000	**−2.73**	−0.234	1.4×10^{-5}
	light	−0.57	−0.115	0.106	**−3.82**	−0.301	4.6×10^{-9}
	moderate	−1.00	−0.114	0.418	**−5.85**	−0.272	8.7×10^{-9}
	severe	**−2.01**	−0.266	5.4×10^{-6}	**−11.63**	−0.489	1.6×10^{-18}

In `medium200/calm` the mean paired difference favours ACO by 0.36 pp while Cliff's delta favours the hybrid. An exploratory check of that cell found that the hybrid wins on 90 of 150 seeds, with a median difference of 0.44 pp in its favour. Four seeds carry differences of 11 to 23 pp the other way, and the paired differences have skewness 2.31. The mean and Cliff's delta answer different questions there, and both are reported.

7.3 Claim 2: adapting the evaporation rate

`aco_adaptive` and `aco_fixed` did not differ significantly in any of the six tested cells. The intervals are wide. At `large1000/severe` the 95% interval on the paired difference is [−1.068, +0.575] pp, so the adaptive colony could be ahead by as much as 1.068 pp (Table 10). The three ACO arms also have by far the worst single steps at 1,000 nodes, with a worst excess of 163.36% for both variants and 164.23% for plain ACO in every scenario. Claim 2 is reproduced as a non-detection. The run does not show that adaptation cannot help.

7.4 Claim 3: allocation control

No sensing policy beat the frozen split by more than a negligible margin (Table 9). P1 was never significant at either tier, and the incremental policy was nominally worse than the frozen split in five of its six tested cells. P2 was significant once, at `large1000/light`, where the direct policy beat the frozen split by 0.18 pp with $\delta = -0.097$ (adjusted $p = 0.0055$). P5 was never significant. No Claim 3 cell was significant at `medium200`.

We read the outcome as the null branch of the pre-registered rules, since only one scenario meets any other branch. The `large1000/light` cell meets the second rule literally, since there the direct policy beat the frozen split but not the timetable. It also meets the fourth, because S1 in that cell favours the direct policy over the incremental one ($\delta = -0.105$, adjusted $p = 0.012$). Both effects are negligible, and they are two of the 34 cells, both in `large1000/light`.

Table 9: Claim 3 at `large1000`. Cliff's delta, with the mean paired difference in pp in brackets. Negative values favour the first arm of the pair. Bold cells are significant after Holm correction. P1 in `calm` is degenerate.

Code	Pair	calm	light	moderate	severe
P1	adaptive vs fixed	–	+0.010 (+0.02)	+0.037 (+0.09)	+0.046 (+0.12)
P2	direct vs fixed	−0.081 (−0.14)	**−0.097** (−0.18)	−0.031 (−0.07)	+0.025 (+0.09)
P5	scheduled vs fixed	−0.047 (−0.10)	−0.052 (−0.08)	−0.048 (−0.10)	−0.058 (−0.16)
P3a	adaptive vs scheduled	+0.047 (+0.10)	+0.063 (+0.09)	+0.082 (+0.19)	**+0.100** (+0.27)
P3b	direct vs scheduled	−0.034 (−0.04)	−0.045 (−0.11)	+0.019 (+0.04)	**+0.080** (+0.24)
S1	direct vs adaptive	−0.081 (−0.14)	**−0.105** (−0.20)	−0.063 (−0.16)	−0.021 (−0.03)

At `large1000/severe` the timetable beat both sensing policies. P3a gave $\delta = +0.100$ (adjusted $p = 0.00045$) and P3b gave $\delta = +0.080$ (adjusted $p = 0.00047$), so the timetable was ahead by 0.27 and 0.24 pp. Both effects are negligible. In an exploratory count, the incremental policy beat the timetable on 47 of 150 seeds in that cell. The P3a effect grows with the disruption rate at 1,000 nodes, from +0.063 to +0.082 and +0.100. This ordering is descriptive and uncorrected, and it does not appear at 200 nodes, where P3a reads −0.036, +0.039 and +0.010.

Table 10 gives the bootstrap bounds at 1,000 nodes. The incremental policy's advantage over the frozen split is at most 0.058 pp in `light` and 0.011 pp in `moderate`, and in `severe` the interval excludes any advantage. At `medium200` the intervals on the hybrid comparisons are much wider, with bounds up to 1.47 pp, so the null there is far less precise.

Table 10: 95% bootstrap intervals on selected non-significant comparisons at `large1000`. Differences are first arm minus second, in pp. The bound is the largest advantage for the first arm the interval allows.

Cell	Mean difference	95% interval	Bound
P1 light	+0.018	[−0.058, +0.097]	0.058
P1 moderate	+0.087	[−0.011, +0.196]	0.011
P1 severe	+0.115	[+0.018, +0.221]	0
P2 moderate	−0.069	[−0.198, +0.057]	0.198
P2 severe	+0.087	[−0.069, +0.246]	0.069
P5 severe	−0.155	[−0.296, −0.026]	0.296
P4 light	−0.018	[−0.463, +0.433]	0.463
P4 moderate	−0.388	[−0.945, +0.163]	0.945
P4 severe	−0.243	[−1.068, +0.575]	1.068

P5 at `severe` is not significant after correction, yet its interval lies entirely on the timetable's side.

A pre-registered check concerned graphs in which one long edge, added when components were joined, carries much of the optimal route. At 200 nodes, 63 of 150 instances carry more than 10% of the initial optimal cost on such an edge. With the seeds split at that pre-set threshold, none of the 19 tested P1, P2, P3a, P3b and P5 cells was significant in either half, for 87 low-bridge and 63 high-bridge seeds. No 1,000-node instance has a joining edge.

7.5 Exploratory analyses

Table 11 shows what the controllers did in `severe`. The frozen arm never moved. The incremental policy moved on 58.2 of 60 steps, travelled 0.343 in total and never rose above an even split. The direct policy asked for far more movement than the governor allowed and was clamped on 44.9 steps, about 75% of the trial. For both sensing policies the applied split was negatively correlated with the recency signal, as designed, though weakly: −0.137 for the incremental policy and −0.196 for the direct one.

The recency signal rises with the disruption rate (Fig. 4). At 1,000 nodes it equals 1.0 on

Table 11: Allocation behaviour in `severe`, mean per 60-step trial. Values agree at both tiers to two decimals.

Arm	Steps moved	Steps clamped	Total travel	Range of f
`hybrid_fixed`	0.0	0.0	0.000	0.500–0.500
`hybrid_adaptive`	58.2	0.0	0.343	0.303–0.500
`hybrid_scheduled`	59.0	5.0	0.790	0.210–0.750
`hybrid_direct`	52.3	44.9	2.567	0.203–0.613

0.0%, 9.5%, 25.9% and 53.9% of steps from `calm` to `severe`, with means of 0.000, 0.253, 0.527 and 0.782. The 200-node values agree within 0.003. In `severe` the signal sits at its ceiling on about half of all steps.

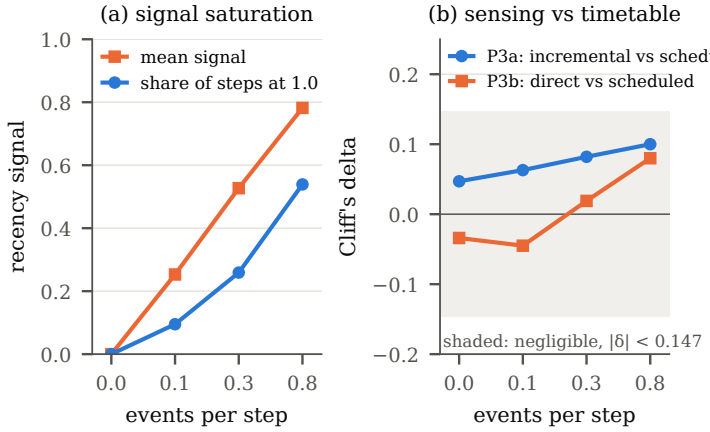


Figure 4: Recency signal and the sensing-versus-timetable effect at `large1000`, by disruption rate. Scenarios are evenly spaced, not to scale. (a) Share of steps at which the signal equals 1.0, and its mean. (b) Cliff's delta for P3a and P3b. Positive values favour the timetable.

Table 12 collects the other exploratory results. Per-trial time on the last three 1,000-node chunks rose from 17.3–18.6 s to 20.3–23.8 s, consistent with thermal throttling, so the timings describe this run and are not hardware-independent benchmarks. Search outcomes depend only on the seed and are unaffected.

Table 12: Supporting analyses (exploratory).

Analysis	Why it was run	Result	What it shows
Recovery, `medium200`	speed of return after a damaging disruption	steps to recover for fixed, adaptive, direct and scheduled: 12.53, 12.32, 11.72, 12.75 in `light`, 7.50, 8.03, 7.40, 7.65 in `moderate`, 4.61, 4.68, 5.05, 4.36 in `severe`, on 87 to 306 damaging events per arm	the controlled hybrids recover at similar speeds
Recovery, `large1000`	the same, on the tier without joining edges	same order: 8.65, 10.05, 14.00, 9.25 in `light` on 12 to 15 events, below the floor, 11.74, 13.47, 13.02, 15.14 in `moderate` on 33 to 36 events, 7.56, 6.55, 8.11, 5.95 in `severe` on 41 to 52 events	no consistent ordering among the controlled hybrids
Steps at optimum	share of steps on an exactly optimal route	12 to 32% at `medium200`, at most 0.59% at `large1000`	the metric carries information only at 200 nodes
Availability	how often an arm had no route	100% of steps at `large1000`, lowest 99.68% for ACO at `medium200`	routes were almost always available

Analysis	Why it was run	Result	What it shows
Route changes	whether moving the split destabilises the route	15.4 to 16.2 changes per trial for the controlled hybrids at `medium200/severe`, and 12.5 for ACO against 2.0 for PSO at `large1000`	the controlled hybrids change route at similar rates
Timing	cost per step	at 1,000 nodes, A* 16.4 ms, PSO 64.7 ms, hybrid 198.6 ms, ACO 285.8 ms, and at 200 nodes 1.5, 24.0, 31.5 and 28.5 ms	A* is exact and 4 to 21 times faster than every timed arm

8 Discussion and Methodological Comparison

8.1 The hybrid result

On the 1,000-node graphs, combining ACO and PSO under one budget beats both components in every scenario, by a wide margin over ACO and a small, consistent one over PSO. Allocation control is not the source of the gain. The uncontrolled 50/50 hybrid already has it, and the controlled hybrids, frozen or moving, stay within 0.28 pp of it. The hybrid shares a budget, returns the better of two routes and exchanges information between its components, and this design cannot say how much each of the three contributes.

At 200 nodes the result against ACO holds only under `severe`. ACO does far better there than at 1,000 nodes. A plausible reason is that the smaller graph is easier for a colony steered by straight-line distance, which leaves less room for the hybrid to improve on it. The qualification matters because every earlier calibration in the project was done at 200 nodes. The advantage over PSO holds at both sizes, while the large advantage over ACO holds only on the larger graphs.

8.2 Precision of the null results

The two nulls differ in precision at 1,000 nodes. For sensing, the intervals hold any benefit of the incremental policy over the frozen split under 0.06 pp, and in `severe` they support none. For evaporation-rate adaptation, the interval in `severe` still admits a benefit of 1.068 pp. The difference most likely comes from the arms themselves. Both ACO variants have mean excess of 12.8 to 14.3% at 1,000 nodes and a worst step above 160%, while the hybrids sit near 3%, so the same 150 seeds resolve the hybrid comparisons far more finely. At 200 nodes the sensing intervals widen to bounds of up to 1.47 pp and lose that advantage. Claim 2 is therefore the weaker of the two negative results, and it is the premise of the allocation approach.

The significant P3a and P3b effects are negligible in size. With 150 paired seeds the design detects differences too small to matter in practice, and the effect sizes show how small they are.

8.3 Explanations for the allocation null

Two explanations can be set aside. The controllers were not inert, since Table 11 shows them moving on nearly every step while the frozen twin never moved. Bridge-dominated graphs are also unlikely to account for the null. The full 200-node set already had no significant Claim 3 cell, so the bridge split mainly confirms that the null does not depend on those instances, and the 1,000-node graphs, which have no joining edges, give the same null. Allocation control did not disturb the search either, as routes were missing on at most 0.32% of steps and route changes and recovery were similar across the controlled hybrids.

Three candidates remain, and the run cannot rank them. The first is the signal. It records that a disruption happened and how long ago, with no measure of how much changed, so a small change and a large one draw the same response. It also saturates as disruptions become more

frequent, although it sits at its ceiling on only about half of the `severe` steps. The second is how far the policies could move. The governor shortened the direct policy's requests on about three quarters of its steps, so that policy never applied its designed response in full, and the incremental policy moved little by design.

The third is the direction of the response. Both sensing policies shift budget toward PSO after a disruption, and in `severe` the incremental policy never rose above an even split. At 1,000 nodes in `severe`, the timetable and the frozen split, which do not react to disruptions, both had nominally lower mean excess than the two sensing policies. The direct policy, which holds f near 0.8 while the graph is calm, had the lowest excess of any arm in `calm` and `light`. If the colony contributes more inside the hybrid than its stand-alone excess suggests, for example through the routes it passes to PSO, moving budget away from it after each disruption would cost quality, and more so as disruptions become frequent. That would fit the growth of the P3a effect with the disruption rate. The pattern breaks at 200 nodes, where the incremental policy had the lowest excess of any arm in `light`. This interpretation is built from the results above and has not been tested. A sweep of fixed splits in each scenario, with a policy that moves the split the other way, would test it.

The tested policies respond to recency only. That choice was recorded in the pre-registration, when the set of arms was frozen so that no policy could be added after a null. The observation already carries the number of edge events at each step, so a magnitude-aware policy would need no change to the harness. Even that count measures the size of a change and not whether the change touched an arm's route. A staleness measure, the pheromone lost on closed edges, captures that directly. In a design-stage trial it was zero on 51 of 63 disrupted steps and stayed within $[0, 0.1]$, so it would need a threshold or a nonlinear policy rather than a linear map. The completed comparisons stand as computed, and their subject is recency-driven allocation.

8.4 Comparison with recent studies

Table 13 sets the controls in Dynaswarm's design against the ten surveyed studies. The table records design coverage, not performance.

Table 13: Controls stated in each report. A dash means not reported, not ineffective.

Study	Dynamic world	ACO and PSO	Exact optimum	Equal work	100+ paired seeds	Multiplicity correction	Recovery
Zhang 2023, HACO	–	✓	–	–	–	–	–
Yang 2023, IPSO-IDWA	✓	–	–	–	–	–	–
Shan 2024, IAACO-DWA	✓	–	–	–	–	–	–
Wang 2024, improved ACO	–	–	–	–	–	–	–
Dao 2024, MOPSO-DWA	✓	–	–	–	–	–	–
Ma 2025, ITACO	–	–	–	–	–	–	–
Li 2025, IEACO	–	–	–	–	–	–	–
Ahmad 2026, ABGPSO	–	–	–	–	–	–	–
Huang 2026, IACO-DWA	✓	–	–	–	–	–	–
Zhang 2026, IACO-QDWA	✓	–	–	–	–	–	–
Dynaswarm	✓	✓	✓	✓	✓	✓	✓

Dynaswarm's coverage follows from its purpose as a controlled comparison and says nothing about which method finds shorter routes. The DWA studies model velocity and collision response, which Dynaswarm does not. In the other direction, none of the ten reports, as far as their text shows, combines the seven controls of the last row.

8.5 Iterations, compute and parameter isolation

An ACO iteration builds one route per ant, and a PSO iteration decodes one route per particle plus up to three repairs for each particle that dead-ends. Equal iteration counts therefore do not mean equal work, and the cost of an iteration also varies with the graph. Dynaswarm fixes $B = 300$ constructions per step for every budgeted arm, which leaves population sizes free while holding the unit of search constant. For the same reason the report makes no claim about iterations to convergence.

All arms ran with the default settings of Section 3, and no parameter was tuned. The controller comparisons change only f_t, with total work, the graph timeline and the scoring held fixed, so a difference between controlled hybrids cannot come from extra work. Tuned components could change the Claim 1 margins, and a sensitivity analysis that counts tuning cost is future work.

9 Individual Contributions and Project Governance

Table 14: Contribution statement.

Member	Contribution
Vaibhav Krishna Bansal	Repository and project integration, mathematical algorithm reference (PSO and ACO), report and presentation integration.
Rudra Rajpurohit	Experimental design, development of the hybrid algorithms, integration of all algorithms and the graph model into the dashboard, choice of the tech stack, execution of the 1,200-trial evidence run.
Kartikey Goyal	Literature survey of 2023 to 2026 ACO and PSO path-planning studies (Table 1), comparison of their evaluation controls with Dynaswarm's (Table 13), review of the statistical method (Wilcoxon test, Holm correction, Cliff's delta thresholds).
Pavit Agrawal	Initial version of the dashboard, preprocessing of the step records into per-trial values, runtime benchmarking of the arms, preparation of the result figures.

9.1 Governance and quality assurance

The comparisons, decision rules and analysis plan were recorded in the project's decision log before the run, with dated amendments. Design decisions are logged apart from results, so policies that failed remain in the record and every pre-registered comparison is reported as computed. Automated checks cover seed ranges, row counts, budgets, route feasibility and the dashboard's edge ledger, which reconciles the initial edges and every addition, reopening, closure and deletion with the current edge set.

10 Limitations and Threats to Validity

10.1 Internal and statistical validity

Fixed timelines and paired seeds remove differences in the world between arms, but an implementation error would affect every arm alike. The exact optimum, the re-pricing of every route, the external meter and the bit-for-bit regeneration of a trial are independent checks against that. The tests and corrections hold for the 34-cell family of each tier and for these 150 seeds. Three reporting decisions followed the run: the recovery floor was applied per arm, it was not applied to `calm`, where no damage can occur, and steps at optimum were read at 200 nodes after proving

nearly constant at 1,000. None of them changes a comparison. The exploratory analyses were also chosen after the run and are used only to interpret the pre-registered results.

10.2 Construct validity

The sensing policies read recency only, so magnitude-aware allocation remains untested (Section 4). The incremental policy moved little because of its small step, and the direct policy was clamped on most steps, so the run tests both policies as configured and governed. Recovery depends on its tolerance and baseline definitions. Counting constructions equalises search work but not wall-clock cost, and the timings come from a host that throttled during the run. A shortest weighted path does not encode curvature, acceleration or collision margins.

10.3 External and reporting validity

Only synthetic random geometric graphs at two sizes were tested, with one 60-step horizon, one budget and four Poisson rates. Extending the findings to road networks, warehouses, grids, adversarial failures or other budgets needs further runs. Each disruption draws up to 25 edge events at both sizes, so it touches a smaller share of the 1,000-node graph, and recovery at 1,000 nodes rests on 12 to 52 damaging events per arm. All arms used default parameters, although the project plan called for per-tier tuning. The PSO is a project-specific encoding, and the hybrid's advantage cannot be split between budget sharing, route selection and information exchange without further ablations. On these instances A* is exact and faster than ACO, PSO and the hybrid, which rules out a deployment claim for the approximate methods. The literature figures in Section 2 come from different maps and protocols and cannot be ranked against each other, and the dashboard screenshots show one seed.

11 Conclusion and Future Work

Dynaswarm asked whether a fixed search budget should be redistributed between ACO and PSO in response to disruption on a changing shortest-path problem. Under equal budgets and identical worlds, combining the two methods helps. The 50/50 hybrid beat PSO everywhere and beat ACO in every 1,000-node scenario, by about 11 percentage points, with a narrower result at 200 nodes. Redistributing the budget from a recency signal did not help. No sensing policy beat the frozen split by more than a negligible margin, and the bounds keep any benefit of the incremental policy at 1,000 nodes under 0.06 pp. At that size, under frequent disruption, a timetable that ignores the graph did slightly better. Adapting ACO's evaporation rate also produced no detectable gain, though the run cannot exclude a gain of about one percentage point.

These results bound recency-driven allocation without settling the question for allocation in general. The controllers moved on nearly every step, which rules out an inert mechanism. The signal carried no measure of how much changed, though, the governor held back the direct policy, and the incremental one moved little by design. The direction of the response was never varied either.

The next experiment follows from that. A policy reading the event count already in the observation, and one driven by a staleness measure, should be pre-registered and compared with recency-only policies on the same 150 paired seeds. The same run should include a sweep of fixed splits per scenario and a policy that reverses the direction of the response. Tuning parameters per tier, scaling the disruption size with graph size and adding road or warehouse graphs would widen the scope. Any claim about routing performance would then need an incremental exact method such as D* Lite as its baseline.

Declarations

Acknowledgement. The authors thank Mrs. Vidhya V for mentoring the project, and the School of Computer Engineering, Manipal Institute of Technology, Manipal Academy of Higher Education.

Funding and competing interests. The work received no external funding. The authors declare no competing interests.

Data and code availability. The repository contains the source code, configuration, tests, the frozen manifest and the derived results record. The 720,000 raw step rows, the run's provenance record and the analysis output are held by the authors, and the rows can be regenerated from commit `b4d3d28` and the listed seeds. The code is released under the MIT License.

Use of AI assistance. Generative AI agents were used throughout the project. They wrote implementation specifications, implemented and tested code under the authors' direction and review, checked analyses, and helped draft and typeset this report. All results come from the recorded evidence run. The authors directed the work, reviewed each change and are responsible for the content.

References

[1] P. E. Hart, N. J. Nilsson, and B. Raphael, "A formal basis for the heuristic determination of minimum cost paths," *IEEE Trans. Syst. Sci. Cybern.*, vol. 4, no. 2, pp. 100–107, 1968, doi: 10.1109/TSSC.1968.300136.

[2] S. Koenig, M. Likhachev, and D. Furcy, "Lifelong planning A*," *Artif. Intell.*, vol. 155, pp. 93–146, 2004.

[3] S. Koenig and M. Likhachev, "D* Lite," in *Proc. 18th AAAI Conf. Artif. Intell.*, 2002, pp. 476–483.

[4] Z. Zhang, J. Lu, Z. Xu, and T. Xu, "Mobile robot path planning based on hybrid ant colony optimization," *J. Intell. Fuzzy Syst.*, vol. 45, no. 2, pp. 2611–2623, 2023, doi: 10.3233/JIFS-231280.

[5] Z. Yang, N. Li, Y. Zhang, and J. Li, "Mobile robot path planning based on improved particle swarm optimization and improved dynamic window approach," *J. Robot.*, vol. 2023, Art. no. 6619841, 2023, doi: 10.1155/2023/6619841.

[6] D. Shan, S. Zhang, X. Wang, and P. Zhang, "Path-planning strategy: Adaptive ant colony optimization combined with an enhanced dynamic window approach," *Electronics*, vol. 13, no. 5, Art. no. 825, 2024, doi: 10.3390/electronics13050825.

[7] H. Wang, S. Wang, and T. Yu, "Path planning of inspection robot based on improved ant colony algorithm," *Appl. Sci.*, vol. 14, no. 20, Art. no. 9511, 2024, doi: 10.3390/app14209511.

[8] T.-K. Dao, T.-G. Ngo, J.-S. Pan, T.-T.-T. Nguyen, and T.-T. Nguyen, "Enhancing path planning capabilities of automated guided vehicles in dynamic environments: Multi-objective PSO and dynamic-window approach," *Biomimetics*, vol. 9, no. 1, Art. no. 35, 2024, doi: 10.3390/biomimetics9010035.

[9] J. Ma, Q. Liu, Z. Yang, and B. Wang, "Improved trimming ant colony optimization algorithm for mobile robot path planning," *Algorithms*, vol. 18, no. 5, Art. no. 240, 2025, doi: 10.3390/a18050240.

[10] P. Li, L. Wei, and D. Wu, "An intelligently enhanced ant colony optimization algorithm for global path planning of mobile robots in engineering applications," *Sensors*, vol. 25, no. 5, Art. no. 1326, 2025, doi: 10.3390/s25051326.

[11] J. Ahmad *et al.*, "Enhancing performance of global path planning for mobile robot through Alpha-Beta Guided Particle Swarm Optimization (ABGPSO) algorithm," *Measurement*, vol. 257, Art. no. 118633, 2026, doi: 10.1016/j.measurement.2025.118633.

[12] L. Huang *et al.*, "Path planning for delivery robots based on an improved ant colony optimization algorithm combined with dynamic window approach," *Sensors*, vol. 26, no. 1, Art. no. 72, 2026, doi: 10.3390/s26010072.

[13] Z. Zhang, X. Bai, H. Yang, and S. Zhang, "Dynamic path planning via enhanced ACO and DWA algorithms," *Appl. Sci.*, vol. 16, no. 2, Art. no. 583, 2026, doi: 10.3390/app16020583.

[14] M. Dorigo, V. Maniezzo, and A. Colorni, "Ant system: Optimization by a colony of cooperating agents," *IEEE Trans. Syst., Man, Cybern. B*, vol. 26, no. 1, pp. 29–41, 1996, doi: 10.1109/3477.484436.

[15] T. Stützle and H. H. Hoos, "MAX-MIN ant system," *Future Gener. Comput. Syst.*, vol. 16, no. 8, pp. 889–914, 2000, doi: 10.1016/S0167-739X(00)00043-1.

[16] M. Mavrovouniotis and S. Yang, "Adapting the pheromone evaporation rate in dynamic routing problems," in *Applications of Evolutionary Computation (EvoApplications 2013)*, LNCS, vol. 7835. Berlin, Germany: Springer, 2013, pp. 606–615, doi: 10.1007/978-3-642-37192-9_61.

[17] J. Kennedy and R. Eberhart, "Particle swarm optimization," in *Proc. IEEE Int. Conf. Neural Netw.*, vol. 4, Perth, Australia, 1995, pp. 1942–1948, doi: 10.1109/ICNN.1995.488968.

[18] F. Wilcoxon, "Individual comparisons by ranking methods," *Biometrics Bull.*, vol. 1, no. 6, pp. 80–83, 1945.

[19] S. Holm, "A simple sequentially rejective multiple test procedure," *Scand. J. Statist.*, vol. 6, no. 2, pp. 65–70, 1979.

[20] N. Cliff, "Dominance statistics: Ordinal analyses to answer ordinal questions," *Psychol. Bull.*, vol. 114, no. 3, pp. 494–509, 1993, doi: 10.1037/0033-2909.114.3.494.

[21] J. Romano, J. D. Kromrey, J. Coraggio, and J. Skowronek, "Appropriate statistics for ordinal level data: Should we really be using t-test and Cohen's d for evaluating group differences on the NSSE and other surveys?" presented at the Annu. Meeting Florida Assoc. Institutional Res., 2006.

[22] B. Efron, "Bootstrap methods: Another look at the jackknife," *Ann. Statist.*, vol. 7, no. 1, pp. 1–26, 1979.